

X-EPIC仿真验证工具P2E

Author: 孙雪健

- 软件目录：147服务器/opt/xepic/hpe-24.09.01.s003
- 内含手册：/opt/xepic/hpe-24.09.01.s003/doc，Quick Start@Chapter 3, **File Templates@Appendix A** 【重要】
- 官方示例：/opt/xepic/nanhu_demo.tgz (Prototyping Mode)
- 注意license只包含vsyn, vcom, vdbg的权限，不包含手册中关于xwave等内容

Preparation

1. 环境配置

```
# X-EPIC
export GALAXSIM_HOME=/opt/xepic/GalaxSim_24.09.00.a055_x64
export GALAXFV_HOME=/opt/xepic/GalaxFV_23.12.00.d478_x64
export FUSION_DEBUG_HOME=/opt/xepic/FusionDebug_24.09.00.a055_c_x64
export HPE_HOME=/opt/xepic/hpe-24.09.01.s003
export VSYN_HOME=${HPE_HOME}
export VCOM_HOME=${HPE_HOME}
export VDBG_HOME=${HPE_HOME}
export VRMCTL_HOME=${HPE_HOME}
export XWAVE_HOME=${HPE_HOME}
export DBGIP_HOME=${HPE_HOME}/share/pnr/dbg_ip
export RLM_LICENSE=5053@ubuntuv80-CS5280H2

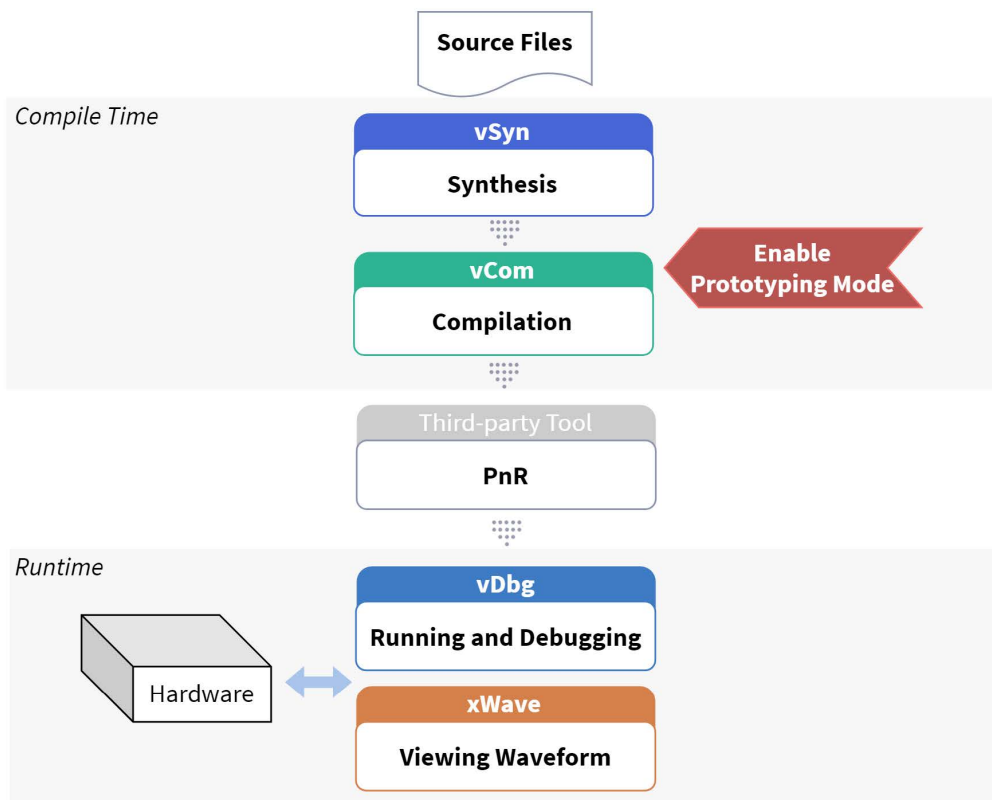
export VIVADO_HOME=/home/data/xilinx/Vivado/2024.2
export
PATH=${GALAXSIM_HOME}/bin:${FUSION_DEBUG_HOME}/bin:${GALAXFV_HOME}/bin:${HPE_HOME}/bin:${HPE_HOME}/xepic_license:${VIVADO_HOME}/bin:$PATH
```

2. 启动license 目录/opt/xepic/hpe-24.09.01.s003/xepic_license下运行./rlm -c HuaPro-Fudan_Univ.lic 若出现(rlm) Cannot bind Web Server port 5054, exiting, 使用ps aux | grep rlm检查是否有正在运行的license，如有则无需重开；或者lsof -nP -iTCP:5053 -sTCP:LISTEN查看5053端口是否被占用。
3. 添加vivado license lic目录/home/data/sunxuejian/xepic/xilinx_lic，打开vivado license manager，添加目录下的所有.lic文件
4. python管理 运行是会调用官方软件目录下的python版本，无需手动添加但要设置好环境避免和本地py冲突。

Workflow

根据需求选择Emulation或Prototyping两种模式

- Emulation是把FPGA当作软硬结合的仿真平台，目的是验证逻辑正确性，会保留JTAG调试核、信号探针等调试接口
- Prototyping模式模拟真实运行场景，会关闭非必要的调试逻辑以减少资源占用并提升运行速度



Synthesis - vSyn

使用官方工具vSyn进行综合生成.vm文件，也可选用Synplify Premier, Vivado等。运行结束后务必确认 **vsyn.log** 文件中无error，且warning不影响关键步骤运行。

基础命令

```
vsyn <your_verilog_srcs> -top <your_top_module> -o <your_.vm> -work
<your_work_lib>
```

可选参数

1. **-f** ./src/filelist.lst
2. **-I** ./src/include
3. **-D** 宏定义
4. **-log**
5. **-area-report**

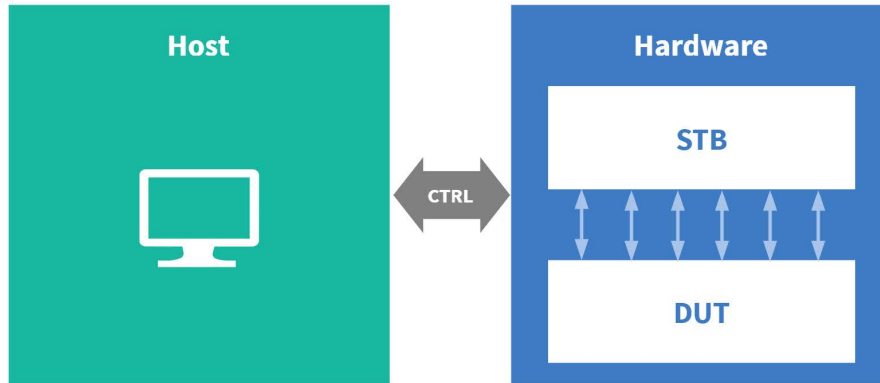
注意事项

1. **【重要】** 由chisel生成verilog文件时需要关注top module io是否是悬空，如悬空应当删掉，否则会在后续步骤中报错/探查不到。

2. 在vsyn过程中可在src当中添加参数（详见手册4.8节），比如(`* syn_ramstyle="block_ram" *`)分配为BRAM等。

STB (Synthesizable testbench)

STB是vsyn支持的一种可在硬件上实时运行、用于验证DUT功能的可综合测试平台。



编写规则主要如下：（详见手册4.11节）

1. 所有测试逻辑必须封装在initial块内
2. initial块内所有事件控制语句必须使用同一个时钟信号和同一边沿
3. STB顶层模块可以包含端口，也可以不包含端口
4. 支持在全局声明时初始化，或在initial块内用阻塞赋值初始化
5. 支持使用连续的`@(posedge clk)`语句描述受多个时钟沿控制的值序列变化，构成一个隐式的有限状态机
6. initial块中不能混合变量初始化和事件控制语句。例如，在`@(posedge clk)`之前的初始化赋值可能在综合后被忽略
7. 对于涉及跨模块引用或非恒定赋值的逻辑，通常必须包含时钟事件语句，否则可能无法综合或行为异常

【重要】 即便完全按照手册的规则编写仍有可能在后续测试时探查失败！另外，STB综合后的逻辑在硬件上运行，其内部状态的调试和观测不如软件仿真灵活，且会占用较多FPGA资源，拖慢PnR过程。所以，可以改写verilog/sv testbench为可综合的形式并设定好探查信号，来代替STB。

Memory and DDR

P2E支持对内存以及DDR的数据进行读写与探查，详见手册Chapter 8。此处结合DDR4 Memory Backdoor的流程简要说明。

1. RTL的标注 给目标内存添加`xpvs_use_xdram`属性，标记为HCM；添加`syn_ramstyle="block_ram"`属性，标记为BRAM等。仅支持单/双端口同步内存；内存最大容量16GB，数据宽度 $\leq 2048\text{bit}$ ，地址宽度 $\leq 31\text{bit}$ ，端口数 ≤ 8 。
2. vcom脚本配置
 1. 确认HCM内存 `memory_options -add {use_xdram top_module.ddd_mem}`
 2. 绑定AXI4接口 `bind_ports -netlist_macro_ports {nm_ddd_ports} -uni_debug_ports {0.A.0 axi4_ddd}`
 3. 配置HDF文件 `CARD += {"type": "CAPC_16G_V1", "connector": "0.A.3"}`
3. vdbg脚本配置
 1. 硬件连接 `set_phc_vol -id 0.A -bank 3 -voltage 1.2`

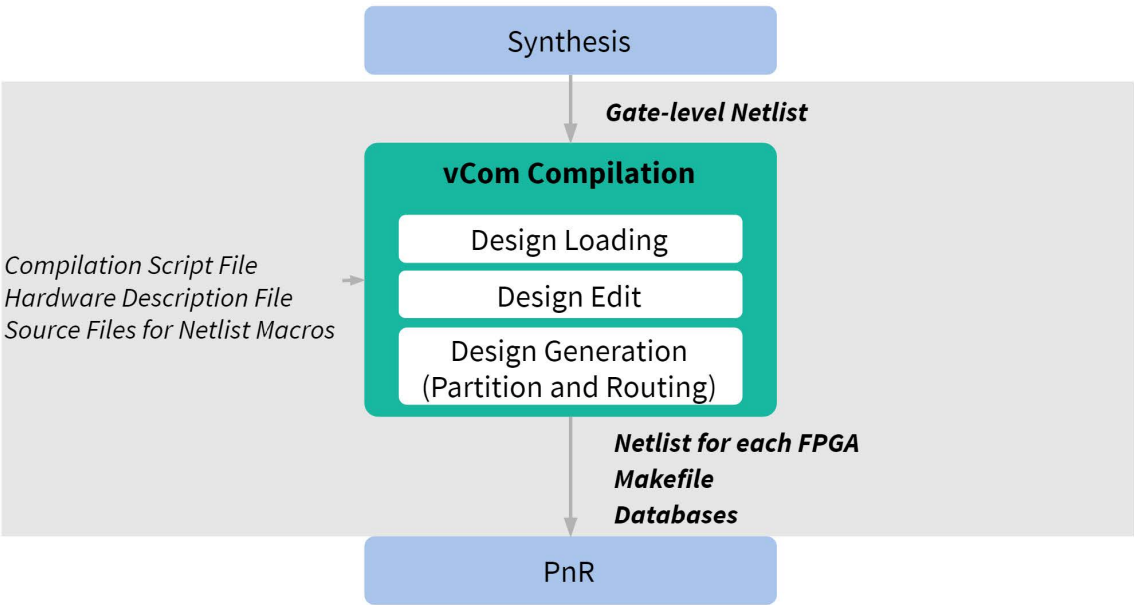
- 2. 写入数据（支持%readmemh或%fread）`memory -write -addr 0x0 -format %readmemh -file ./ddr_data.txt -mem top_module.-ddr_mem`
- 3. 读取数据`memory -read -addr 0x0 -len 1024 -format %readmemh -file ./read_back.txt -mem top_module.-ddr_mem`
- 4. 数据文件处理，可以是纯数据文件，也可以添加一定的参数。此处给出手册中的例子。

```
Memory name: <memory_name>
Format: %b|%o|%h
Start address: <start_address>
End address: <end_address>
<data>
```

Compiling - vCom

参考官方示例以及下方的表格编写.hdf文件以及vcom_compile.tcl，而后运行vcom vcom_compile.tcl(Emulation Mode)或者vcom -prototyping_mode vcom_compile.tcl(Prototyping Mode)；完成后务必查看vcom.log是否存在可能影响后续步骤的warning和error。

P2E不会自动添加波形监测，关于想要添加波形监测的信号，请参考下文vdug - gui - waveform部分以及手册P351说明，在vcom脚本当中添加合适的trace_net -add <net>或者streaming_trace_net -add <net>命令。注意，添加physical probe会极大拖慢pnr时间，因此需要精简添加的监测信号数量。



HDF File

HDF是P2E仿真平台的硬件配置文件，用来告知工具P2E板卡的基础硬件信息、建立连接、分配资源的必要配置，无此文件会导致工具无法识别硬件。关于DDR子板连接问题请参阅手册5.7.7节。注意，只需指定第一个PHC连接器。例如，如果指定了连接器0.A.3，则连接器0.A.4和0.A.5也会被占用，无法用于任何其他目的。

仅用作逻辑验证场景时，.hdf文件配置如下。

```
VERSION += 1.0
DEVICE += P2E
BOARD += {"index": 0, "type": "FOUR_CHIP_BOARD", "CLK": 0, "IP":
"10.134.141.148"}
```

【重要】 欲添加Physical probe，则必须添加DDR子板连接；欲添加Streaming Probe，则必须添加HAC子板连接（暂缺）。目前实验室板卡支持的DDR connector位于0.A.3以及0.B.3。

下面是一个使用Physical probe的示例。

```
CARD += {"type":"CAPC_16G_V1", "connector":"0.A.3"}
```

Emulation常用命令/参数对照表（手册P107）

命令/参数组合	核心功能
<code>design_read -netlist <file_list></code>	读取vSyn/第三方工具生成的网表（.vm/.v），支持多文件加载，可指定-work参数关联库
<code>design_load -top <top_module></code>	加载顶层模块，建立设计入口，关联已读取的网表文件
<code>emulator_spec -add "file <HDF>"</code>	加载硬件描述文件（HDF），定义板卡类型、FPGA分区、线缆连接等硬件配置
<code>emulator_util -add {<location> <max>}</code>	分配FPGA分区资源利用率（如{0.A 70}表示0.A分区最大利用率70%），支持按资源类型精细配置
<code>netlistmacro -instance <path> -package</code>	配置网表宏（DDR/PCIe等），绑定宏实例到指定FPGA分区，关联宏包路径
<code>netlistmacro_param -module <name></code>	为网表宏模块绑定参数（如NL_FILE_PATH指定.dcp文件），确保IP核物理适配
<code>create_clock -sig_name <sig> -frequency</code>	定义用户时钟，指定信号名和频率（如100Mhz），emulation模式下自动映射到全局时钟资源
<code>trace_net -add <sig></code>	标记信号为物理探针，支持波形捕获，可指定-instance/-depth扩展实例下信号捕获范围
<code>write_net -add <sig></code>	标记信号为可写，支持vDbg runtime强制force, release操作
<code>read_net -add <sig></code>	标记信号为可读，支持vDbg runtime读取信号值（get_value），默认顶层输出端口自动设为read_net
<code>memory_access -add <mem></code>	标记DUT内存为可后门访问，支持vDbg读写内存数据，第三方网表需配合classify_mem.py生成映射文件
<code>terminal_keep -add <sig></code>	保留未分配终端信号，避免编译优化删除，用于vdbg调试或外部交互
<code>design_edit</code>	启动设计编辑流程，执行优化、时钟转换、环路破除等编译操作

命令/参数组合	核心功能
<code>design_generation</code>	生成FPGA分区网表和PnR脚本，多FPGA设计自动完成分区和路由

Prototyping常用命令/参数对照表（手册P104）

命令/参数组合	核心功能
<code>cable_connection -def {}</code>	绑定调试线缆到FPGA物理引脚PHC
<code>terminal_assign -add {}</code>	保留信号用于vdbg调试/波形捕获
<code>create_clock -name/-period/-waveform</code>	创建时钟，指定周期/波形
<code>assign_clock -add {GCx your_clk_sig}</code>	将逻辑时钟绑定到FPGA全局时钟资源
<code>set_clock_groups -asynchronous</code>	标记异步时钟组，避免跨时钟域时序检查
<code>global_reset -net/-duration_time</code>	配置全局复位，指定复位信号/持续时间/同步时钟； <code>-assert 0</code> 表示低复位， <code>-assert 1</code> 表示高复位
<code>elim_common_logic -enable</code>	启用逻辑优化
<code>create_isa -clock/-label</code>	创建ISA调试域，指定采样时钟/标签
<code>isa_net -add {your_area your_sig} -type</code>	将信号加入ISA调试域； <code>-type trace_and_trigger</code> 支持跟踪+触发，无type仅跟踪

常见问题

- 1. 【重要】 对所有Top Module的io务必不要使用`terminal_keep -add`，会覆盖掉自动生成的read/write net
- 2. 如果核心目标是用vdbg复现dut的逻辑，而非需要绑定所有IO的物理连接时，在`terminal_keep -add`当中无需保留所有信号，可以加快vcom编译速度。比如mant_0_0_1和mant_0_0_0属于同类型数据位，验证了mant_0_0_0能正常传输，就说明整个数据通路是通的，无需重复保留。
- 3. 模式选择很重要，注意报告当中的Read/Write Nets，Emulation模式下这两者不会是0

Running FPGA PnR

这步较慢，进入vCom生成的fpgaCompDir目录下，`make all`，生成bit流
fpgaCompDir/part_b*_f*/pnrDir/*.bit *.bin、时序/资源报告
fpgaCompDir/part_b*_f*/pnrDir/rptDir/、日志文件xepic_pnr_summary.log,
<fpga_top>_pnr_summary.rpt及辅助分析文件。

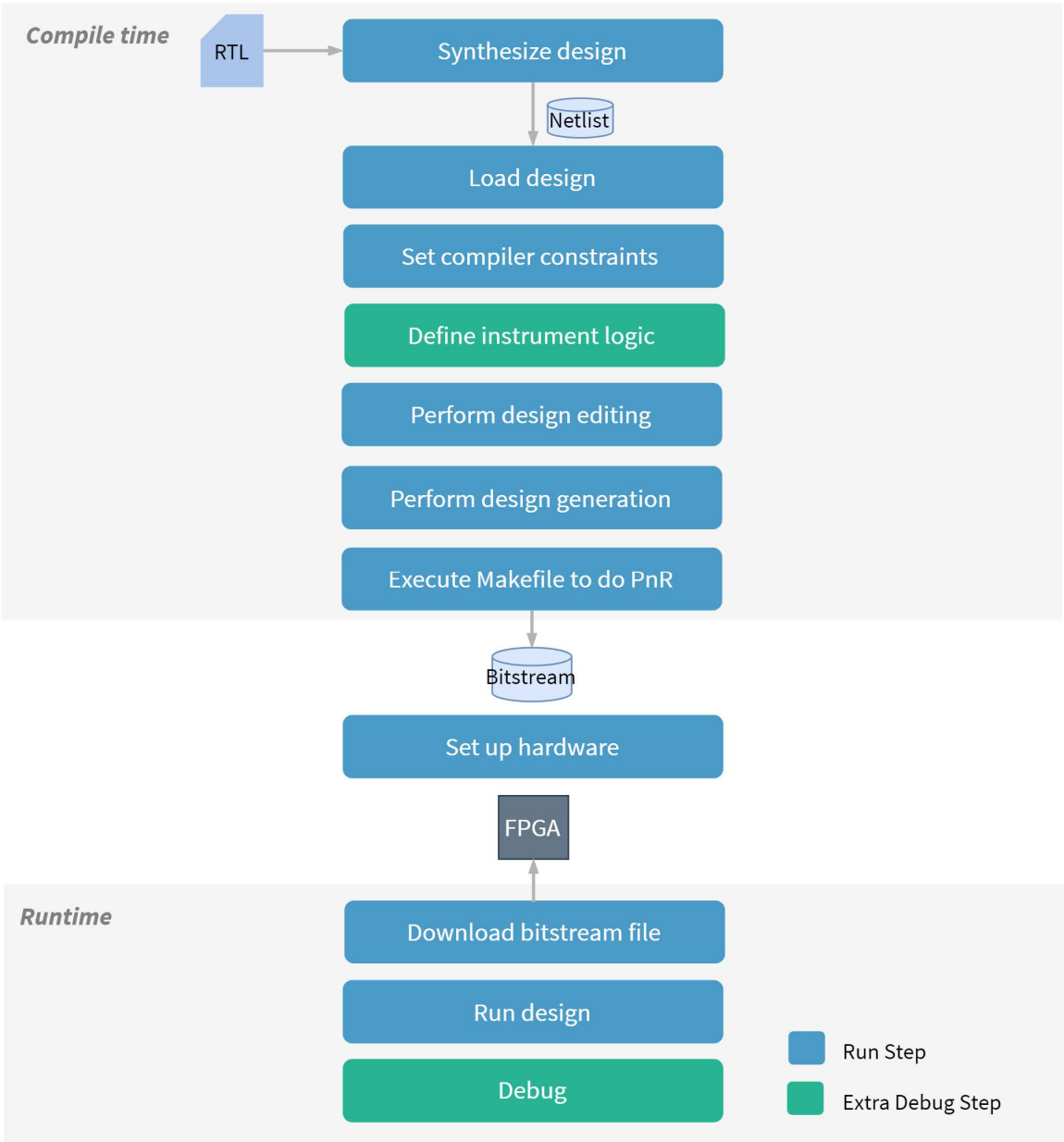
常见问题：

- 1. 【重要】 P2E工具默认要求所有IO端口必须指定物理位LOC和电气标准IOSTANDARD（用于实际板卡部署），若使用场景是纯逻辑验证（通过vdbg驱动DUT，无需绑定真实物理引脚），这些约束完全不必

要，只需禁用对应的DRC，在<your_prj>/PNR_settings.tcl当中添加约束即可。这个文件需要先运行make all之后才会生成。

```
ena_pre_opt_cons_check_stop_PNR 0
set_property SEVERITY {Warning} [get_drc_checks NSTD-1]
set_property SEVERITY {Warning} [get_drc_checks UCIO-1]
```

Running on Hardware

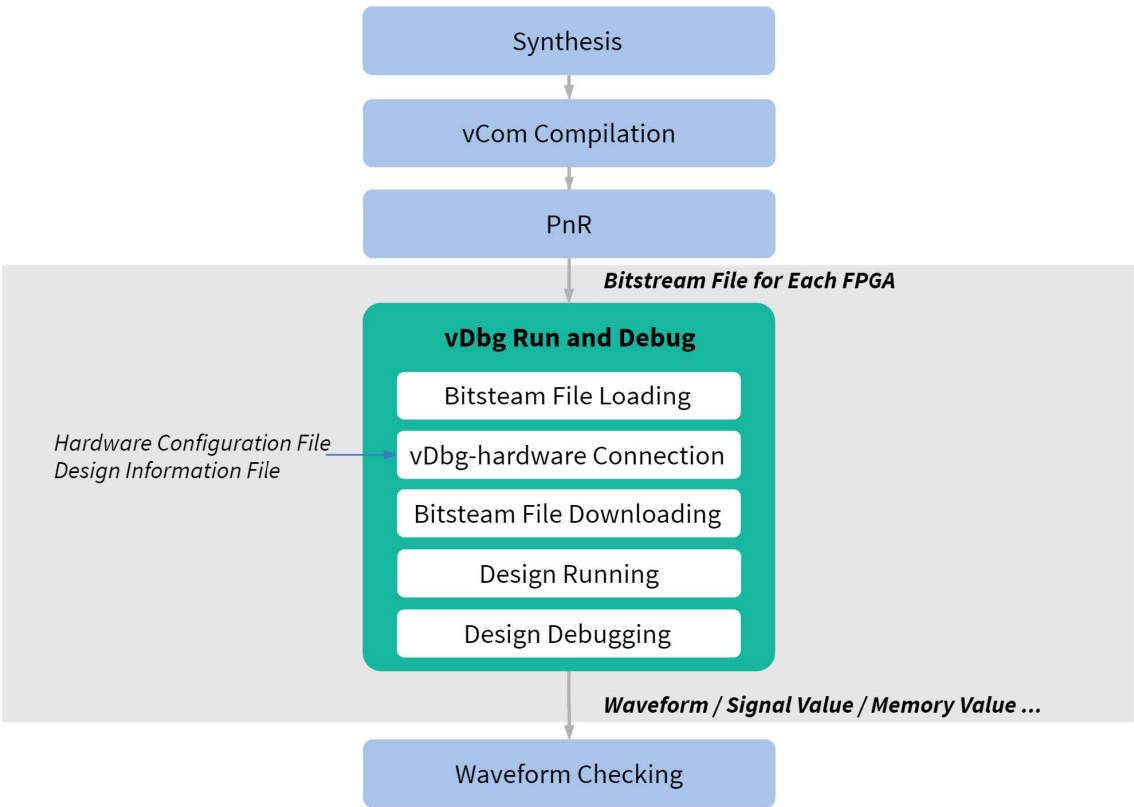


Debugging - vDbg

首先需参考[常见问题1](#)测试板卡连接，再参考下表或手册7.3节以及testbench的逻辑编写vdbg.tcl。

- tcl语法可以参考[此链接](#)，可以参考/home/data/sunxuejian/xepic/DTSA/src/HardwareSimulation_w1_IC1_ICLANE32_OC32_BIT4.sv与/home/data/sunxuejian/xepic/DTSA/vdbg.tcl的转换关系

- 注意force/probe等探查/赋值操作须在Emulation模式下完成



常用参数对照表（手册P257）

命令与参数	核心功能
<code>design .</code>	加载当前目录下vcom编译生成的比特流/网表等产物
<code>set_clock -clock GC2 100Mhz</code>	绑定全局时钟资源（需与vcom编译时的assign_clock配置一致）
<code>hw_server .</code>	连接本地P2E硬件服务器
<code>hw_server -release</code>	释放硬件服务器连接（测试结束后释放资源）
<code>download</code>	将编译好的比特流下载到FPGA
<code>force <your_sig> <your_data></code>	强制驱动DUT信号值，等同于Verilog的信号赋值
<code>get_value <your_sig></code>	读取DUT信号当前值
<code>run <your_cycle>rclk</code>	推进时钟周期
<code>capture -file <your_file> -net <your_sig></code>	批量捕获信号值并保存到文件（Verilog的\$fwrite）
<code>open/close <your_file_path></code>	读写文件
<code>tracedb -close</code>	清理vdbg运行过程中的临时跟踪数据）
<code>exit</code>	退出vdbg环境

这里针对physical probe的操作给出一个示例

```
tracedb -enable
tracedb -open tracing_done -vcd -overwrite
trace_signals -add MatMulTopTest.done_flag
run 1500rclk
tracedb -upload
tracedb -close
```

说明

1. 所有的信号名需完整，如`Top.io_input_inst_in_valid`，不可省略顶层名；
2. 可结合`while`循环 + `probe -get`实现握手逻辑，举个例子：

```
while {[get_value MatMulTop.io_input_inst_in_ready] != 1} {run 1rclk}
```

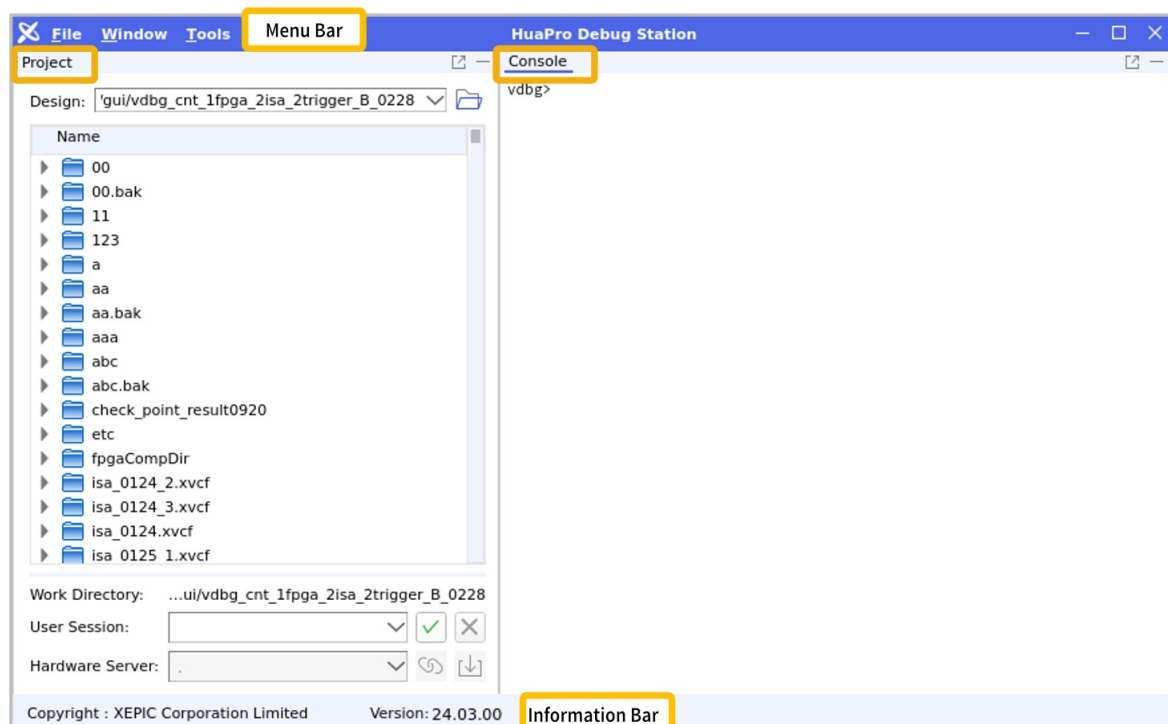
常见问题

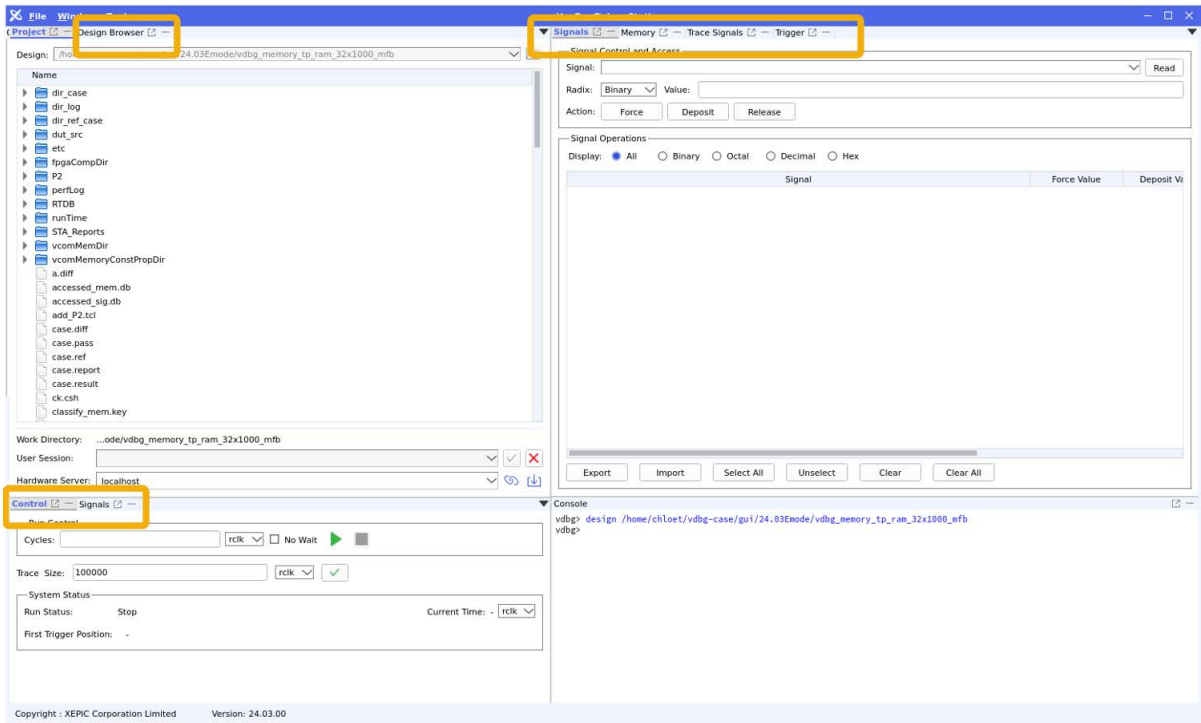
1. **【重要】** 测试板卡连接
 - Step 1: `ping 10.134.141.148`
 - Step 2: 确认`etc/vdbg/config.json`当中`hw_list`当中包含了板卡正确的ip
 - Step 3: vdbg下输入`hw_server` . 确认板卡已连接

GUI and Waveform

这部分在手册Chapter 12有更详细的配图说明。

- 输入命令`vdbg -gui`启动图形界面。
- vDbg GUI 核心组件包括：





组件	核心功能
Project 面板	选择设计路径、配置用户会话、加载设计文件
Control 面板	控制设计运行（指定周期/模式）、查看系统状态（运行状态/当前时间）
Signals 面板	信号强制/沉积/释放/取值、批量导入/导出信号操作
Memory 面板	内存后门读写、指定地址范围/文件格式、导出读写操作
Trace Signals 面板	选择探针类型（物理/流式）、生成/上传波形、批量导入探针
Trigger 面板	加载ATT触发器模块、实例化绑定信号、启用触发器
Design Browser 面板	查看设计层级树、拖放信号/模块名快速输入
Console 面板	执行Tcl命令、查看操作反馈日志

主要操作流程

1. 加载与下载设计
 - （可选）在Project面板点击文件夹图标切换设计路径
 - 输入用户会话名（可选，文件会保存在 xxx.session 目录），点击Apply加载设计
 - 在Hardware Server区域点击连接图标
 - 在Console面板设置时钟频率、PHC 电压
 - 点击「Download」图标，将比特流下载到硬件
2. 运行设计控制
 - 1. 在 Control 面板配置运行参数：
 - 指定运行周期（如 1000 rclk），点击Run启动
 - 勾选No Wait进入非阻塞模式（需清空Cycles字段）
 - 2. 停止运行

- 达到指定周期自动停止
- 触发器触发后自动停止
- 手动点击Stop图标

3. 信号操作（Signals面板）

1. Force

- 输入信号名（支持 AAA[7:4] ）或从下拉列表选择；
- 选择进制（二进制/八进制/十进制/十六进制），输入值
- 点击Force，信号会保持该值直到被释放

2. Deposit

- 操作步骤同Force，点击Deposit
- 信号仅保持该值一个时钟周期，之后由设计驱动

3. Release

- 输入信号名或在Signal Operations表格选择目标信号
- 点击Release，信号恢复由设计驱动

4. 内存读写（Memory 面板） 以下操作均支持Export与Import的批量操作

1. 内存写入

- 输入内存完整路径（可通过 Console 执行 `memory -list` 查看）
- 指定地址范围（Start/End）
- 选择写入模式（32位十六进制/全0/全1/递增0/递增1）或选择文件
- 选择文件格式（如 `%readmemh`），点击Write

2. 内存读取

- 输入内存完整路径和地址范围；
- 选择读取类型（仅支持backdoor访问）
- 选择保存文件路径和格式，点击Read

5. Waveform（Trace Signals 面板） 以下操作支持Import的批量操作

1. 物理探针（需DDR4子板）

- 选择探针类型Physical
- 输入波形数据库名，选择格式（XEDB/CWDB/VCD），点击Apply创建
- 点击Select选择要追踪的信号，添加到探针列表
- 在 Control 面板设置 Trace Size（追踪内存大小），然后运行
- 点击Upload上传波形数据，完成后点击Disable关闭探针

2. 流式探针

- 选择探针类型Streaming（仅支持 XEDB 格式）；
- 后续操作同物理探针，波形数据会自动上传

6. 触发器配置（Trigger 面板）

- 加载 Verilog 触发器模块文件 (.v) ， 点击Read
- 输入实例名，选择模块，绑定输入端口信号（按模块端口顺序填写）， 点击Add实例化
- 点击Enable启用触发器，调整Post Trigger Samples（触发后运行周期占比）
- 运行设计，触发器触发后会自动停止时钟，可查看信号/内存数据

7. 波形查看

- 在 Trace Signals 面板点击Wave Viewer，直接调用 Fusion Debug 打开波形
- 或点击菜单栏 **Tools > Fusion Debug**，手动加载 XEDB 波形文件

补充说明

1. 部分操作依赖编译时配置：如物理探针需编译时添加 **trace_net**、内存读写需添加 **memory_access**
2. 批量操作（信号/探针/内存）的文本文件需保证信号名完整、格式无错误